

Assignment 4

Working with Files Streams and Serialization

Objective: build a file-processing pipeline that parses cold-chain sensor readings, queries adjacent values, and creates a JSON archive without leaking streams or hiding file access inside lazy sequences.

1 Task

Create a console application named ColdChainMonitor. It imports temperature readings from a text file, finds unsafe readings and abrupt changes, writes a JSON archive, then reads the archive back. Estimated time: 150–180 minutes.

1.1 Input file

Create readings.txt with the following lines in exactly this order:

```
S1|2026-09-22T08:10:00Z|Cold|5.2
S2|2026-09-22T08:01:00Z|Frozen|-18.0
S1|2026-09-22T08:00:00Z|Cold|4.0
S1|2026-09-22T08:05:00Z|Cold|8.7
S2|2026-09-22T08:06:00Z|Frozen|-12.0
S1|2026-09-22T08:15:00Z|Cold|8.0
S3|bad-date|Cold|3.5
S2|2026-09-22T08:11:00Z|99|-17.0
S1|2026-09-22T08:20:00Z|Cold|5.5
S1|2026-09-22T08:25:00Z|Frozen|-17.0
```

1.2 Required processing

- Read the file through a Stream. Do not use File.ReadAllLines or File.ReadAllText.
- Parse every non-empty line and keep an error with its original 1-based line number when parsing fails.
- Detect range violations and abrupt changes by comparing adjacent valid readings of the same sensor.
- Serialize the completed archive to archive.json and deserialize it again for verification.

2 Data model and parsing

2.1 Required types

Use immutable records for stored data and a custom enumeration for the storage class.

```
public enum StorageClass { Cold, Frozen }

public sealed record Reading(
    string SensorId,
    DateTimeOffset Timestamp,
    StorageClass StorageClass,
    decimal Temperature);

public sealed record ImportError(
    int LineNumber, string RawLine, string Message);
```

Define Alert and MonitoringArchive records yourself. The archive must contain all valid readings, all import errors, all alerts, and a CreatedAtUtc value supplied by Program.cs. Do not call DateTime.Now inside parsing or analysis methods.

2.2 Parsing contract

Each non-empty line has exactly four pipe-separated fields:

```
sensorId|timestamp|storageClass|temperature
```

- Trim every field. SensorId must be non-empty.
- Parse Timestamp exactly as yyyy-MM-dd'T'HH:mm:ss'Z' and normalize it to UTC.
- Parse StorageClass case-insensitively, but reject undeclared numeric values such as 99.
- Parse Temperature with CultureInfo.InvariantCulture and only AllowLeadingSign | AllowDecimalPoint.
- A comma decimal such as 5,5 is invalid; NaN and infinity are not valid decimal values.
- After its first valid reading, a sensor must keep the same StorageClass. A mismatch becomes an ImportError.
- Sensor IDs are compared with StringComparer.OrdinalIgnoreCase.

2.3 Parsing separation

Keep line parsing independent from file access. A pure TryParseReading function should parse one line and return success or an error. The importer adds line numbers and applies the per-sensor StorageClass consistency rule.

2.4 Valid readings from the supplied file

Lines 1–6 are valid. Lines 7–10 are invalid for four different reasons. The valid readings remain in original input order in the archive.

3 Streams and adjacent queries

3.1 Stream ownership

Implement these operations or equivalent methods with the same ownership behavior:

```
static ImportResult ReadReadings(Stream input)
static void WriteArchive(Stream output, MonitoringArchive archive)
static MonitoringArchive ReadArchive(Stream input)
```

- The caller owns each supplied Stream. Your methods must leave it open.
- A method reads or writes from the stream's current Position. It must not silently rewind the stream.
- Dispose every StreamReader and StreamWriter that you create, using `leaveOpen: true`.
- `ReadReadings` must finish reading before it returns. Do not return a lazy sequence backed by a reader or stream.
- `Program.cs` owns `FileStream` instances and disposes them with `using`.

3.2 Temperature rules

STORAGE CLASS	SAFE MINIMUM	SAFE MAXIMUM
COLD	2.0	8.0
FROZEN	-22.0	-15.0

- The limits are inclusive. Exactly 8.0 for Cold and exactly -15.0 for Frozen are safe.
- A reading is outside range when it is below the minimum or above the maximum for its class.
- An abrupt change occurs when the absolute difference from the previous valid reading of the same sensor is greater than 4.0.
- The first valid reading of each sensor has no previous reading and cannot be abrupt.

3.3 Adjacent values

Group valid readings by `SensorId`, sort each group by `Timestamp`, and only then compare adjacent pairs. You may use `Zip` with `Skip(1)`, an index-based query, or a small reusable `Pairwise` function.

```
ordered.Zip(
    ordered.Skip(1),
    (previous, current) => (previous, current));
```

Create at most one Alert per current reading. If a reading is both outside range and abrupt, store both flags or both reasons in that one alert.

3.4 Deterministic order

Store alerts ordered by `SensorId` using `OrdinalIgnoreCase` and then by `Timestamp`. This makes console output, JSON, and tests deterministic.

4 Serialization and debugging tasks

4.1 JSON requirements

- Use System.Text.Json. BinaryFormatter and third-party serializers are not allowed.
- Use camelCase property names and write enum values as strings with JsonSerializer.Converters.Add(new JsonStringEnumConverter()).
- Write indented JSON. Deserialization must reject a null archive.
- After serialization to a MemoryStream, explicitly set Position = 0 before deserialization.
- Verify the round trip by comparing values and collection contents, not object reference identity.

4.2 Lazy data backed by a disposed reader

```
static IEnumerable<string> ReadLines(Stream input)
{
    using var reader = new StreamReader(input, leaveOpen: true);
    return ReadAll(reader); // ReadAll is a lazy iterator.
}
```

4.3 Reading from the end of a stream

```
using var stream = new MemoryStream();
JsonSerializer.Serialize(stream, archive, options);

var restored =
    JsonSerializer.Deserialize<MonitoringArchive>(stream, options);
```

4.4 Comparing unrelated sensors

```
var ordered = readings.OrderBy(r => r.Timestamp);
var pairs = ordered.Zip(
    ordered.Skip(1),
    (previous, current) => (previous, current));
```

4.5 Required explanation

- Why the returned sequence can depend on a reader that has already been disposed, and why materialization fixes it.
- Why deserialization starts at the current Position and what must happen before reading a just-written MemoryStream.
- Why globally adjacent readings can belong to different sensors, and where GroupBy must occur.

5 Expected results and tests

5.1 Expected result

ITEM	EXPECTED VALUE
INPUT LINES	10
VALID READINGS	6
IMPORT ERRORS	4: bad timestamp, numeric enum, comma decimal, class mismatch
OUTSIDE-RANGE READINGS	2: S1 at 08:05; S2 at 08:06
ABRUPT CHANGES	2: S1 4.0 → 8.7; S2 -18.0 → -12.0
ALERTS	2; each affected reading has both reasons
SAFE BOUNDARY	S1 at 08:15 with 8.0 is safe

5.2 Required alert order

SENSOR	TIMESTAMP UTC	TEMPERATURE	REASONS
S1	08:05	8.7	Outside range; abrupt change
S2	08:06	-12.0	Outside range; abrupt change

5.3 Automated tests

Write at least 14 automated tests. The following eleven are mandatory:

1. the supplied file produces 6 valid readings, 4 errors, and 2 alerts;
 2. numeric enum text 99 is rejected even when Enum.TryParse succeeds;
 3. 5,5 is rejected under invariant parsing;
 4. a StorageClass change for the same sensor becomes an import error;
 5. Cold 8.0 and Frozen -15.0 are safe boundary values;
 6. adjacent comparisons never cross sensor boundaries;
 7. input order may differ from timestamp order without changing the alerts;
 8. ReadReadings leaves a caller-owned stream open;
 9. WriteArchive leaves a caller-owned stream open;
 10. a MemoryStream round trip preserves archive values and string enum names;
 11. enumerating stored results after the source stream is disposed still works.
- Choose at least three additional boundary, malformed-input, or stream-position cases.

6 README and submission

6.1 README.md

Answer briefly and in your own words:

1. Who owns each stream, and why do your helper methods leave it open?
2. Why is a lazy sequence tied to a StreamReader unsafe after the method returns?
3. Why must MemoryStream.Position be reset before reading the bytes just written?
4. Why are readings grouped before adjacent pairs are created?
5. Why is Enum.IsDefined still needed after Enum.TryParse?
6. Which parts of the pipeline are pure and which parts perform I/O?
7. Why does the JSON round-trip compare values instead of reference identity?
8. How do deterministic ordering and an injected CreatedAtUtc improve tests?

6.2 Deliverables

12. A compilable .NET console project named ColdChainMonitor.
13. readings.txt and the generated archive.json.
14. README.md with the required answers and debugging explanations.
15. Automated tests in a separate test project.
16. Archive name: IT-25XX_Surname_Name_Assignment4.zip.

7 Grading

CRITERION	POINTS
FILE ACCESS AND STREAM OWNERSHIP	2
PARSING AND CUSTOM ENUMERATION	2
ADJACENT-VALUE QUERY AND ALERT CORRECTNESS	2
JSON SERIALIZATION AND VERIFIED ROUND TRIP	2
DEBUGGING EXPLANATIONS AND AUTOMATED TESTS	2
ORAL DEFENSE	5
TOTAL	15

7.1 Mandatory grading conditions

- No oral defense: 0/15.
- The project does not compile: maximum 5/15.
- The project compiles but cannot process the supplied file: maximum 7/15.
- A criterion that the student cannot explain during defense receives 0 points.
- Using File.ReadAllLines or File.ReadAllText: 0 points for file access and stream ownership.
- Returning lazy results that fail after the reader is disposed: 0 points for file access and stream ownership.
- Comparing adjacent readings across different sensors: 0 points for the adjacent-query criterion.
- Writing JSON without reading it back and checking values: maximum 1/2 for serialization.
- A declared technique that is bypassed by the actual processing path receives no credit.
- Copied code or AI-generated code that the student cannot explain: 0/15.
- When several limits apply, the lowest maximum grade is used.